

Technical Report: Plant Disease Classification

Sevak Chteyan

1. Introduction

This report details the development of a robust computer vision system designed to identify plant diseases across diverse host species. By addressing significant class imbalance and the "host-shortcut" bias, we achieved a peak Accuracy of **0.86** and mAP of **0.94** using a **ConvNeXtV2 Tiny** architecture.

2. Dataset Challenges

The dataset comprises 7,783 training images across 32 plant species and 39 diseases, forming 82 unique plant-disease combinations. Class distribution is characterized by a significant long-tail imbalance, with majority classes exceeding minority classes by an order of magnitude. To ensure robust evaluation, the validation set was standardized to 10 samples per disease ($n=390$); however, this small sample size necessitates caution regarding potential overfitting. While source images exhibit heterogeneous resolutions (typically 400–1000 px, with outliers exceeding 5,000 px) and minor aspect ratio variance, all inputs were Resized to a uniform **384×384** resolution to maintain pipeline consistency.

3. Methodology

We benchmark three backbone architectures: **EfficientNetV2-Small**, **DINOv3 ViT Small Plus**, and **ConvNeXtV2-Tiny**. All models are trained using cross-entropy loss with label smoothing.

The data pipeline applies **RandomResizedCrop (384×384)**, **RandomHorizontalFlip**, and **TrivialAugmentWide**, combined with batch-level **MixUp** and **CutMix**. These augmentations are designed to mitigate shortcut learning (e.g., plant-specific cues) and improve robustness to variations in viewpoint, scale, and texture.

In the final configuration, weighted sampling is disabled. Instead, class imbalance is addressed implicitly through augmentation and label smoothing, avoiding aggressive resampling strategies that may amplify noise and lead to overfitting.

Further details on hyperparameters and training configuration are provided in the **Appendix**.

3.1 EfficientNetV2 Small

We adopt a two-stage training strategy to stabilize optimization and improve generalization.

In the first stage, MixUp and CutMix are disabled to avoid excessive regularization during early training, which can introduce noise and hinder convergence (see Figure 1). Batch Normalization layers are frozen to preserve pre-trained feature statistics and prevent instability caused by noisy mini-batch estimates. The model is trained for 10 epochs with a total batch size of 64, using a learning rate of 5×10^{-4} with step-wise decay (factor 0.1 every 3 epochs). Regularization includes Dropout and stochastic depth with a survival probability of 0.8.

In the second stage, the backbone is unfrozen and the model is fine-tuned for 15 epochs.

The learning rate is linearly warmed up from 0 to 3×10^{-5} for the backbone and 3×10^{-4} for the classification head, followed by cosine annealing.



Figure 1. Effect of **MixUp/CutMix** on early training dynamics, showing increased training noise and slower convergence in the initial epochs.

3.2 DINOv3 ViT Small Plus

We follow a similar two-stage training strategy. In the first stage, the backbone is frozen and the model is trained without MixUp/CutMix to ensure stable adaptation of the classification head. In the second stage, the backbone is unfrozen and fine-tuned end-to-end.

Training uses cross-entropy with label smoothing, along with standard augmentations and batch-level MixUp/CutMix during fine-tuning. A lower learning rate is applied to the backbone compared to the head, with linear warmup followed by cosine decay.

3.3 ConvNeXtV2 Tiny

In contrast to previous models, ConvNeXtV2-Tiny is trained end-to-end from the start. We employ **layer-wise learning rate decay** with a base learning rate of 3×10^{-4} and a decay factor of 0.9, assigning lower learning rates to earlier layers to preserve pre-trained representations while allowing deeper layers to adapt.

Training follows the same setup as above, including cross-entropy with label smoothing,

standard augmentations, and batch-level MixUp/CutMix. A cosine learning rate schedule with warmup is used for optimization.

4. Ablations

4.1 Exponential Moving Average

We evaluated Exponential Moving Average (EMA) of model weights as a stabilization technique during training. While EMA is commonly beneficial in large-scale or long-training regimes, it did not yield improvements in our setting.

We attribute this to several factors. First, the training schedule is relatively short, limiting the benefit of weight averaging. Second, strong regularization (MixUp/CutMix, label smoothing, stochastic depth) already stabilizes optimization, reducing the marginal gain from EMA.

Overall, EMA did not improve mAP or accuracy and was therefore excluded from the final configuration.

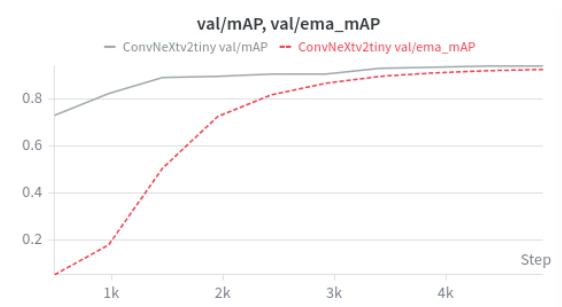


Figure 2. **Validation mAP vs. EMA mAP**. The plot demonstrates the characteristic lag of the EMA during a short training cycle. Because the EMA weights are a cumulative average, they remain heavily influenced by early, low-performing iterations. In this short-regime setting, the training concludes before the EMA can "catch up" to and smooth the final optimized weights, resulting in a lower final mAP compared to the standard model.

4.2 DINOv3 linear probing

We evaluate a linear probing setup for DINOv3, where the backbone is frozen and only a linear classification head is trained. This setting tests the quality of pre-trained representations without task-specific adaptation.

While linear probing achieves reasonable performance, it is consistently outperformed by full fine-tuning (see Figure 3). This suggests that, despite strong pre-training, adaptation to domain-specific features (e.g., disease patterns independent of plant type) is necessary for optimal performance.

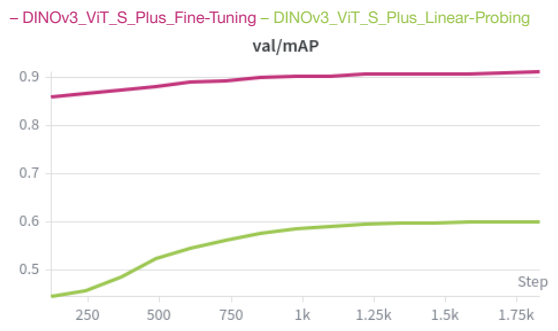


Figure 3. Linear Probing vs. Full Fine-Tuning for DINOv3. The comparison shows a significant performance gap (approximately 30% mAP) between linear probing and fine-tuning. While the frozen DINOv3 backbone provides a solid representative baseline, the substantial gains from fine-tuning confirm that the model benefits heavily from weight updates to specialize in plant-specific pathology.

5. Results

Model	mAP	Accuracy	Params (M)
EfficientNetV2 Small	0.87	0.81	21.5
DINOv3 ViT Small Plus	0.91	0.83	28.7
ConvNeXtV2 Tiny	0.94	0.86	28.6

Table 1, summarizes the performance of the three candidate architectures

We compared three architectures EfficientNetV2-S, DINOv3 (ViT-S+), and ConvNeXtV2-Tiny. Performance was measured using mean Average Precision (mAP). As shown in Table 1, **ConvNeXtV2 Tiny** achieved the highest performance across all metrics, reaching an **mAP of 0.94** and an **accuracy of 0.86**. Notably, it outperformed the DINOv3

model despite having a nearly identical parameter count (~28.6M).



Figure 4. Validation Loss (up) and mAP (down) across training steps. ConvNeXtV2 Tiny (grey) demonstrates superior long-term optimization, achieving the lowest loss and highest mAP. While DINOv3 (pink) shows high zero-shot-like performance at step 0, it lacks the fine-grained scaling seen in the ConvNeXtV2 architecture.

Appendix

This appendix provides implementation details, including model configurations, training and fine-tuning recipes.

To account for the distinct optimization profiles of each architecture, we employed specific fine-tuning recipes as shown in Tables 2,3,4 and 5. We found that extending the training duration was critical for maximizing the accuracy of the ConvNeXtV2-Tiny backbone.

config	value
optimizer	AdamW
base learning rate	5e-4
weight decay	0.01
optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.999$
batch size	64
learning rate schedule	step(3,0.1)
training epochs	10
augmentation	TrivialAugmentWide()
label smoothing	0.1
drop path	0.1

Table 2. **Stage 1 (frozen backbone) training setting for EfficientNet v2 Small and DINO v3 ViT S Plus models.**

config	value
optimizer	AdamW
head lr	3e-4
backbone lr	3e-5
weight decay	0.01
optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.999$
batch size	128
learning rate schedule	cosine decay
warmup epochs	3
training epochs	15
augmentation	TrivialAugmentWide()
label smoothing	0.1
mixup	0.8
cutmix	1.0
drop path	0.1
ema	0.999

Table 3.. **Fine-Tuning setting for EfficientNet v2 Small model.**

For the ConvNeXtV2-Tiny backbone, we implemented Layer-wise Learning Rate Decay utilizing the timm library. Following the strategy proposed by Liu et al., we adopted a granular layer-wise approach rather than a

group-wise one, assigning a distinct decay factor to each individual layer

config	value
optimizer	AdamW
base learning rate	3e-4
weight decay	0.05
optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.999$
layer-wise lr decay	0.9
batch size	64
learning rate schedule	cosine decay
warmup epochs	2
training epochs	10
augmentation	TrivialAugmentWide()
label smoothing	0.1
mixup	0.8
cutmix	1.0
drop path	0.2
head init	0.001
ema	0.9999

Table 4. **End-to-end fine-tuning setting for ConvNeXt v2 Tiny model.**

config	value
optimizer	AdamW
head lr	5e-4
backbone lr	2.5e-5
weight decay	1e-4
optimizer momentum	$\beta_1, \beta_2 = 0.9, 0.999$
batch size	128
learning rate schedule	cosine decay
warmup epochs	3
training epochs	20
augmentation	TrivialAugmentWide()
label smoothing	0.1
mixup	0.8
cutmix	1.0
drop path	0.1

Table 5. **Fine-tuning setting for DINOv3 ViT S Plus model.**